Lesson V

Arrays

Class Objectives

In this lesson, you will learn about:

- Declaring arrays of the elementary data types: `char`, `int`, `float`
- Reading values into the elements of arrays
- Writing values from the elements of arrays
- Searching through the data items in an array
- Sorting the data items in an array

Important Notes:

- **You should type in all the programs in this handout, and run them more than once with different data**
- **You should read, and understand everything in this handout, the material in it forms the basis of the quizzes**
- **If you don't understand something; ask me to explain it again**

Arrays

An array is a set of data items, referred to as *elements*, and all the elements in that set of data items have the same *type*, (`int`, `char`, `float` etc.). In an array, each element is referred to by the name of the array, and distinguished by an element number enclosed between square brackets. The basic form of an array declaration is:

```
<data type> <variable name>[number of elements];
```

Below are example array declarations of type: `char`, `int`, `float`, with a pictorial representation of each.

```
char customer_name[15];
```

customer_name	F	r	a	n	k		S	m	i	t	h				
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]

```
int daily_high_temperatures[370];
```

daily_high_temperatures	72	74	73	75	76	72	77	...				
	[0]	[1]	[2]	[3]	[4]	[5]	[6]		[365]	[366]	[367]	[368]	[369]

```
float london_gold_closing[35];
```

london_gold_closing		292.90	289.90	288.50	0	0	288.80	...	275.75	276.75			
	[0]	[1]	[2]	[3]	[4]	[5]	[6]		[30]	[31]	[32]	[33]	[34]

As you can see from the pictorial representations above, the first element of each array is element[0], and the last element of each array is element[size - 1].

Since [size -1] could be many thousands, putting data into an array using `gets()`, or `scanf()` isn't very appealing.

The standard way to access array elements sequentially, is to use the loop counter variable, in either the `for()`, `while()`, or `do – while()` loops.

Program (`array1.c`)

This first program is a very simple program. All it does, is assign the value of the `for()` loop's loop counter variable into each of the array elements. Then, using another `for()` loop, run through the array again while printing the values in each element to the screen.

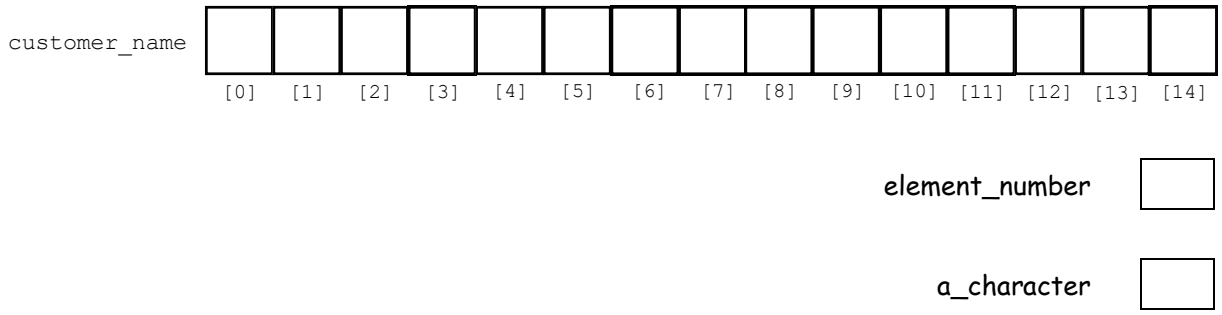
```
/*
  Program "array1.c"
  Written by: Joe Dorward
  Date: 04/12/00

  This program declares an array of integers.
  The loop counter of a for() loop is used to assign values to
  the elements of the array.
  Another for() loop prints the values of the elements to the
  screen.
*/

#include <stdio.h>

void main(void)
{
int loop_counter,
    integer_array[25];

  /* assign value of loop counter into the array */

  for (loop_counter = 0; loop_counter < 25; loop_counter++)
  {
    integer_array[loop_counter] = loop_counter;
  }

  /* print the values of the array to the screen */

  for (loop_counter = 0; loop_counter < 25; loop_counter++)
  {
    printf("\n The value in element: %2d,",loop_counter);
    printf(" is: %d.",integer_array[loop_counter]);
  }
}
```

Program (`cust1.c`)

This program is based on the character array:

```
char customer_name[15];
```

The “eight year old” design



Put 0 into the box “element_number”

Ask for the (first/next) character of the customer’s name

Put the character into the box “a_character”

While character in the box “a_character” isn’t the newline character (start loop)

- Put the character in the box “a_character” into box [element_number] of “customer_name”
- Ask for the (first/next) character of the customer’s name
- Add 1 to the value in the box “element_number”

(end loop)

Put 0 into the box “element_number”

While the number in the box “element_number” is less than the number of characters in “customer_name” (start loop)

- Tell me character in the box number of “customer_name” that is the same as the number in the box “element_number”
- Add 1 to the value in the box “element_number”

(end loop)

The program:

```
/*
  Program cust1.c
  written by: Joe Dorward
  Date: 04/09/00

  This program reads characters for a customer name into
  a character array, then writes them out to the screen.
*/

#include <stdio.h>
#include <string.h>

void main(void)
{
char a_character,
     customer_name[15] = "";

int element_number = 0;

  printf("\n Please enter the customer\'s name: ");

  a_character = getchar();   /* read in the first character */

  while (a_character != '\n')
  {
    customer_name[element_number] = a_character;
    a_character = getchar();
    element_number++;
  }

  element_number = 0;   /* reset value to zero */

  printf("\n The customer\'s name is: ");

  while (element_number < strlen(customer_name))
  {
    printf("%c",customer_name[element_number]);
    element_number++;
  }
}
```

This program will really read in any line of characters, one character at a time, up to the maximum number of characters 15.

Change that 15 to any integer value, and you have a program that will read in that number of characters.

Program (`temps1.c`)

This program is based on the integer array:

```
int daily_high_temperatures[370];
```

As you can see, in this array, there are 370 elements available to hold integers. The name of the array suggests that each element will hold the daily high temperatures. In this example program, I've changed it to a 10 element array to save you from typing in all the 365 integers for the day time high temperatures for the year.

Using round numbers for the number of the elements in an array, prevents problems caused by your programs running into the end of the array, while running through it using a loop.

The program:

```
/*
  Program temps1.c
  written by: Joe Dorward
  Date: 04/09/00

  This program reads in 7 integers, as daily high temperatures for a week.

  Then it searches, and prints out, the highest temperature for the week.
*/

#include <stdio.h>

void main(void)
{
int element_number,   /* declare a loop counter */
    highest_temperature = 0,
    daily_high_temperatures[10];   /* declare an array */

  /* Read the 7 temperatures into the array */

  for (element_number = 0; element_number < 7; element_number++)
  {
    printf(" Please enter the days highest temperature: ");
    scanf("%d",&daily_high_temperatures[element_number]);
  }

  /* Find the highest temperature */

  for (element_number = 0; element_number < 7; element_number++)
  {
    if (daily_high_temperatures[element_number] > highest_temperature)
    {
      highest_temperature = daily_high_temperatures[element_number];
    }
  }

  /* Print out the highest temperature */

  printf("\n This weeks highest temperature, was: ");
  printf("%d",highest_temperature);
}
```

If you compare this program, with the program `high.c`, you will see that the part of the program that finds the highest value is very similar.

Program (gold1.c**)**

This program is based around the float array:

```
float london_gold_closing[35];
```

In it, I have shown you another way of putting values into an array; assignment.

This is exactly the same method as initialization, with which you are already familiar, and it saves you typing in the values each time you run the program.

Zero values are entered for the days when no trading took place.

This program is fairly simple, it asks the user to enter a price for gold, then tells the user how many days the gold price closed above that price.

Date	Closing Price
March 1, 2000	292.90
March 2, 2000	289.90
March 3, 2000	288.50
March 4, 2000	
March 5, 2000	
March 6, 2000	288.80
March 7, 2000	289.75
March 8, 2000	291.20
March 9, 2000	290.50
March 10, 2000	290.25
March 11, 2000	
March 12, 2000	
March 13, 2000	290.65
March 14, 2000	288.75
March 15, 2000	289.15
March 16, 2000	287.35

Date	Closing Price
March 17, 2000	283.90
March 18, 2000	
March 19, 2000	
March 20, 2000	285.75
March 21, 2000	285.15
March 22, 2000	287.80
March 23, 2000	284.75
March 24, 2000	284.85
March 25, 2000	
March 26, 2000	
March 27, 2000	284.15
March 28, 2000	280.65
March 29, 2000	279.75
March 30, 2000	275.75
March 31, 2000	276.75

The program:

```
/*
  Program gold1.c
  written by: Joe Dorward
  Date: 04/09/00

  This program assigns values into an array for the daily price
  of gold in London.

  Then it asks for a price from the user, and it tells the user
  how many days the price of gold closed in London higher than that.
*/

#include <stdio.h>

void main(void)
{
int element_number,   /* loop counter */
    day_counter = 0;   /* counts the number of days */

float entered_price,
      london_gold_closing[35];   /* enough for a month */

  /* ================================================================ */
  /* Assign the prices into the array */

  london_gold_closing[1] = 292.90f;   london_gold_closing[2] = 289.90f;
  london_gold_closing[3] = 288.50f;   london_gold_closing[4] = 0f;
  london_gold_closing[5] = 0f;        london_gold_closing[6] = 288.80f;
  london_gold_closing[7] = 289.70f;   london_gold_closing[8] = 291.20f;
  london_gold_closing[9] = 290.50f;   london_gold_closing[10] = 290.25f;
  london_gold_closing[11] = 0f;       london_gold_closing[12] = 0f;
  london_gold_closing[13] = 290.65f;  london_gold_closing[14] = 288.75f;
  london_gold_closing[15] = 289.15f;

  /* ================================================================ */
  /* Ask for a price, and report */

  printf("\n Please enter the bottom price: ");
  scanf("%f",&entered_price);

  for (element_number = 1; element_number <= 15 ; element_number++)
  {
    if (london_gold_closing[element_number] > entered_price)
    {
      day_counter++;
    }
  }

  printf("\n London gold closed above: $%3.2f,",entered_price);
  printf(" %d days this month.",day_counter);
}
```

From the previous page, enter the rest of the values for March, and run the program.

Program list

- In these programs, you must use arrays, loops, and `if()` statements
- Some of the loops are `for()` loops, others are `while()` loops
- You should do any three of these programs, and submit the programs along with an "eight year old" design for each

Name	Program specification (Write a program that...)	Look at

`temps2.c`	Asks for seven day time high temperatures, and prints out the highest, lowest temperature for the week:
	
	`This week's highest high temperature was: 73 degrees`
	`This week's lowest high temperature was: 67 degrees`
		`temps1.c`
		`low.c`

`cars1.c`	Ask the user to enter the number of passengers in each car that stop at a toll booth. When a negative number is entered, the program prints the number of vehicles, and number of people that have passed through the toll booth:
	
	`There were: 16 cars.`
	`With a total of: 24 passengers`
		`temps1.c`
		`total.c`

`gold2.c`	Prints out all the closing prices:
	
	`Closing Gold for March  1, was: $292.90`
	`Closing Gold for March  2, was: $289.90`
	`Closing Gold for March  3, was: $288.50`
	
	`Closing Gold for March 30, was: $275.75`
	`Closing Gold for March 31, was: $276.75`
	
	The program should only print out the days where the closing price isn't zero. Then extend the program further, so that it prints out the month's highest, and lowest (non zero) closing price:
	
	`Highest closing price was: March  1, at: $292.90`
	`Lowest closing price was: March 30, at: $275.75`
		`gold1.c`

`words.c`	Ask the user to type in a paragraph of text (say the program specification for `cars.c`). It then prints out the number of words in the paragraph, and the number of sentences:
	
	`There are: 41 words`
	`There are:  2 sentences`
	
	A paragraph is only a large character array. If you count the spaces (" ") characters, you would have a close count on the number of words. If you count the periods ("`.`") characters, you will have a close count of the number of sentences.
		`gold1.c`